

To Query or Not to Query: An Empirical Study of Server-State Management Trade-Offs in React

Martin Mosemann

SAP SE

Walldorf, Germany

s_mosemann24@stud.hwr-berlin.de

Sebastian Schlesinger

Berlin School of Economics and Law

Alt-Friedrichsfelde 60, 10315 Berlin, Germany

sebastian.schlesinger@hwr-berlin.de

Abstract—Modern frontend systems increasingly suffer from subtle defects caused by incorrect server-state handling, including duplicated requests, inconsistent UI state after mutations, and hard-to-reproduce race conditions. While query libraries such as TanStack Query and SWR are widely adopted to address these issues, there is little controlled empirical evidence guiding when their adoption is actually justified.

We present a controlled study comparing a custom `useEffect`-based approach with TanStack Query and SWR across twelve representative scenarios. Our results show that query libraries systematically prevent recurring classes of defects—most notably in request deduplication and mutation lifecycle handling—while providing no measurable benefit in simple read-only scenarios.

Based on these findings, we derive actionable decision guidelines that help engineering teams determine when the additional abstraction of a query library reduces risk and complexity—and when it introduces unnecessary overhead in terms of setup cost and conceptual complexity.

Index Terms—server-state management, React, TanStack Query, SWR, empirical study, software maintainability, frontend engineering

I. INTRODUCTION

Incorrect server-state handling is a frequent source of subtle production defects in modern frontend systems, including redundant network requests, inconsistent UI state after mutations, and race conditions that are difficult to reproduce. A recurring engineering challenge in these applications is *server-state management*: fetching, caching, synchronizing, and invalidating data that originates from remote APIs [1]. Unlike local UI state, server state is inherently asynchronous, can become stale without notice, may be requested concurrently by multiple components, and must be kept consistent after mutations.

The default React mechanism for handling side effects, the `useEffect` hook [2], provides no built-in support for caching, deduplication, or retry logic. Each concern must be implemented manually, leading to fragmented, error-prone code as applications grow in complexity. Dedicated query libraries—most prominently TanStack Query [1] and SWR [3]—address these gaps by providing a declarative, key-based cache layer with built-in deduplication, background revalidation, and structured mutation lifecycles.

Despite widespread industry adoption of these libraries, the academic literature lacks controlled, artifact-based comparisons that quantify their trade-offs against well-implemented

native solutions. Practitioner-oriented comparisons exist primarily as blog posts and documentation examples, which are neither peer-reviewed nor systematically designed. A literature search in ACM Digital Library and IEEE Xplore confirmed the absence of studies that quantitatively compare native `useEffect`, TanStack Query, and SWR under common controlled scenarios.

This paper presents an empirical study that fills this gap. Our contributions are:

- A controlled experimental framework isolating the impact of server-state management strategies
- Evidence that query libraries eliminate specific defect classes (e.g., duplicate requests, inconsistent optimistic updates)
- An empirically grounded trade-off model describing when abstraction overhead outweighs benefits
- Actionable decision guidelines for frontend teams

II. BACKGROUND AND PROBLEM CONTEXT

A. Server-State Challenges

Server state differs fundamentally from local client state in five recurring dimensions [1]: (1) an asynchronous pending/success/error lifecycle, (2) staleness—data fetched at time t may be outdated at time $t + \Delta t$, (3) concurrent deduplication when multiple components request the same resource simultaneously, (4) post-mutation cache synchronization, and (5) resilient retry and failure recovery for transient failures.

In practice, these challenges compound: a mutation must invalidate related cache entries, concurrent fetches should be deduplicated, and failed requests should be retried—all while keeping the UI consistent.

B. Existing Approaches

Native `useEffect` [2] executes side effects after rendering and is the standard mechanism for data fetching in React function components. The typical pattern manages `data`, `isLoading`, and `error` state manually; caching, deduplication, and mutation synchronization must be built from scratch.

TanStack Query [1] maintains a centralized, key-based cache with configurable stale times, automatic request deduplication, and structured mutation hooks supporting optimistic updates and rollback.

TABLE I
COMPARISON SCENARIOS AND PRIMARY EVALUATION CRITERIA

ID	Scenario	Primary Criterion
S1–S2	Task list / detail fetch	Cache, dev. effort
S3–S5	Create / update / delete	Mutation handling
S6	Optimistic completion toggle	Optimistic update, roll-back
S7	Paginated / infinite list	Advanced scenarios
S8	Dependent query (user→tasks)	Advanced scenarios
S9	Manual refetch	Cache, revalidation
S10–S11	Error simulation / retry	Error & retry handling
S12	Background revalidation	Cache, revalidation

SWR [3] implements the stale-while-revalidate pattern (RFC 5861 [5]): stale data is served immediately while background revalidation proceeds. Its API is intentionally minimal; mutation coordination relies more heavily on manual cache manipulation.

The decision between these approaches involves trade-offs that are hard to evaluate without controlled experimentation. A native approach avoids external dependencies but requires familiarity with race conditions, in-flight deduplication, and cache invalidation [1]. Query libraries address this complexity systematically but introduce setup overhead and API learning cost.

III. STUDY DESIGN

A. Artifact Description

The comparison artifact is a React + Vite + TypeScript application that implements twelve server-state scenarios three times—with a custom `useEffect`-based baseline, TanStack Query (v5), and SWR (v2)—against a shared in-process mock API. The mock API simulates all CRUD operations with configurable latency (250–700 ms) and three error modes (none, single-next, random at 20% rate with 75% retryable). A centralized request logger records endpoint, parameters, outcome, and concurrent-duplicate flag for each request.

The **native baseline** uses two reusable helper modules: a module-scoped Map-based cache (42 LOC) and a `useNativeQuery` hook (136 LOC) wrapping `useEffect/useState` for loading/error/data state, staleness checks, retries, and optional background refresh—deliberately a lightweight custom abstraction, not raw `useEffect`.

B. Scenarios

Table I lists the twelve scenarios and their primary evaluation criteria. The scenarios cover one or two representative cases per server-state challenge: read scenarios (S1–S2) for cache and deduplication, mutation scenarios (S3–S6) with the highest complexity variance between approaches, and one scenario each for pagination, dependent queries, manual refetch, error recovery, and background revalidation.

TABLE II
EVALUATION CRITERIA WITH EVIDENCE TYPES

Criterion	Evidence	ISO 25010
Setup complexity	Quantitative	Maintainability
Development effort	Mixed	Maintainability
Code structure	Qualitative	Modularity
Cache / revalidation	Mixed	Perf. efficiency
Mutation handling	Mixed	Reliability
Advanced scenarios	Structural	Maintainability
Runtime behavior	Quantitative	Perf. efficiency

TABLE III
CODE VOLUME PER IMPLEMENTATION

Impl.	LOC	Files	Note
Native	818	3	incl. 178 LOC reusable helpers
TanStack Query	626	1	incl. provider & key factory
SWR	546	1	incl. config wrapper

C. Evaluation Criteria

Seven criteria, derived from the server-state challenges and operationalized with predefined advantage thresholds, map to ISO 25010 sub-characteristics [4] (Table II). Anchoring criteria in an independent quality model reduces the risk of circular evaluation.

Our controlled design—shared mock API, identical scenarios, predefined thresholds—isolates the effect of the server-state management approach itself, providing the internal validity necessary to draw causal conclusions about structural trade-offs.

IV. RESULTS

A. Setup Complexity and Development Effort

The native approach requires no packages but 178 LOC of reusable infrastructure before the first scenario can be implemented. TanStack Query and SWR each require one package and a provider wrapper (~10–15 LOC). Table III summarizes code volume; per-scenario effort converges for simple queries but diverges significantly for mutations and pagination.

Table IV presents complementary structural metrics. The native implementation requires twice as many `useState` calls, five times more imperative `catch` blocks, and five additional state variables for pagination alone—all of which library hooks abstract entirely.

B. Cache, Deduplication, and Revalidation

The native cache suppresses re-fetches only when a fresh entry exists. It does not track in-flight requests: two concurrently mounted components with the same key both issue a network request, confirmed by the duplicate-observer scenario (S1) logging two simultaneous `getTasks` calls. TanStack Query deduplicates concurrent subscribers to an in-flight query automatically; the same scenario produced one request and two subscribers. SWR’s 1500 ms deduplication interval achieves the same result. For single-subscriber

TABLE IV
STRUCTURAL METRICS FROM CODE INSPECTION

Metric	Native	TanStack	SWR
useState calls	14	7	7
Imperative catch blocks	5	1	2
Pagination state variables (S7)	5	0	0
Explicit mutation tracking vars	1	0	0

scenarios (S2–S9), all three approaches produced identical request counts. Both libraries provide background revalidation via `refetchInterval/refreshInterval`; the native implementation requires an explicit `setInterval`.

C. Mutation Handling and Optimistic Updates

Each native mutation handler spans approximately 20–30 LOC including cache invalidation and error handling. The optimistic toggle (S6) applies a local state update manually, issues the API call, and restores the previous value in a catch block. TanStack Query’s `useMutation` lifecycle encapsulates optimistic writes (`onMutate`), automatic rollback (`onError`), and post-settlement invalidation (`onSettled`), ensuring no step is omitted regardless of how many mutation call sites exist. SWR achieves the same outcome through imperative composition: the optimistic write, API call, and rollback are sequenced manually—more error-prone than the structured lifecycle but producing less boilerplate than the native approach for non-optimistic mutations.

D. Advanced Scenarios and Runtime Behavior

For pagination (S7), the native implementation requires five state variables and a manual accumulation function. `useInfiniteQuery` and `useSWRInfinite` compress this into a single callback, substantially more compact. For dependent queries (S8), all three implementations are comparable in complexity. Under the random error mode (S10–S11), all implementations were configured for up to two retries. The native retry loop is sequential and per-instance; TanStack Query centralizes retry logic with exponential backoff; SWR applies a configurable flat retry interval globally.

Taken together, these results show that the primary benefits of query libraries are not incremental improvements, but the systematic elimination of recurring and hard-to-diagnose implementation errors.

V. DISCUSSION

A. Interpreting the Trade-Offs

The results reveal a clear pattern: *the benefit of query libraries scales with scenario complexity*. For simple, single-subscriber read scenarios, the native baseline is functionally equivalent—the reusable `useNativeQuery` hook handles loading state, staleness checks, and retries adequately. The divergence emerges in three areas:

Deduplication. The native cache’s lack of in-flight tracking is a correctness issue, not merely an efficiency concern: redundant requests can cause UI flickering, race conditions,

and unnecessary backend load in production. Both libraries eliminate this class of defect by design.

Mutation lifecycle. TanStack Query’s structured `onMutate/onError/onSettled` lifecycle prevents a common class of bug: forgetting to roll back optimistic updates on failure, or omitting cache invalidation after a successful mutation. The native approach relies on developer discipline at every call site—a pattern that scales poorly across teams and codebases.

Pagination. The five additional state variables required by the native approach represent cognitive load and a maintenance burden that grows with each paginated endpoint. Library abstractions eliminate this entirely.

Between the two libraries, TanStack Query provides greater structural discipline and is preferable when mutation workflows are complex. SWR produces fewer lines of code overall and suits read-heavy applications where mutation orchestration requirements are limited. The practical difference between them is most apparent in the optimistic update scenario (S6); for straightforward data fetching, it is marginal.

These results suggest that server-state management should be treated as a cross-cutting architectural concern rather than a local implementation detail. This shift has implications for system design, as decisions about server-state management affect correctness, performance, and maintainability across the entire application.

B. When NOT to Use a Library

Our results also identify conditions under which a query library adds overhead without proportional benefit:

- **Very small applications** with one or two endpoints and no shared server state across components.
- **Applications using meta-frameworks** (e.g., Next.js, Remix) with integrated data loading that already handles caching and revalidation.
- **Teams maintaining a mature in-house abstraction** that already addresses deduplication and cache invalidation.

C. Risks of Naive Implementations

A correct native implementation requires familiarity with race conditions, in-flight deduplication, cache invalidation, and retry strategies—concerns that manifest late and under hard-to-reproduce conditions. Without relevant experience, developers risk building solutions that appear correct under simple conditions but produce silent failures under production load. Libraries address this systematically through type-safe APIs that guide developers through edge cases.

As a further observation consistent with our findings—but not directly tested—library benefits are expected to increase with application complexity: the more components share data and write operations propagate across views, the more structural advantages outweigh setup cost.

VI. IMPLICATIONS FOR PRACTICE

Based on our findings, we offer the following guidelines for engineering teams:

- **Adopt a query library when** the application has multiple components consuming the same server resources, complex mutation workflows requiring cache synchronization, or paginated data access patterns.
- **Prefer TanStack Query when** mutation safety is critical, multiple mutation call sites exist, and the team values structured lifecycle guarantees over API minimalism.
- **Prefer SWR when** the application is read-heavy, mutation orchestration needs are limited, and the team values a smaller API surface.
- **A native approach suffices when** the application has few endpoints, no shared server state across components, and the team has strong expertise in asynchronous state management patterns.
- **Consider team expertise:** for teams with limited experience in asynchronous state management, adopting a library is not merely a productivity optimization but a systematic safeguard against hard-to-diagnose defects.
- **Do not over-invest in custom infrastructure.** If a native implementation begins to require in-flight deduplication and global invalidation broadcasting, the team is effectively rebuilding what a query library provides out of the box.

VII. THREATS TO VALIDITY

Internal validity. All three implementations were created by a single developer, introducing a systematic confound for effort-related findings. The 80-LOC difference between TanStack Query and SWR could reflect implementation style; the larger native-library gap is more robust but cannot fully exclude varying familiarity. LOC is a proxy for code volume, not a direct measure of implementation time or debugging effort.

External validity. The mock API eliminates confounding variables but cannot reproduce production failure modes or server-push invalidation.

Construct validity. The native baseline uses a custom hook, not raw `useEffect`. A more complete in-house abstraction could close most observed gaps; the comparison is between a lightweight custom layer and two established libraries, not between unstructured code and mature tooling.

These threats are deliberate design trade-offs: the controlled setup sacrifices external validity for the internal validity needed to identify structural differences.

VIII. CONCLUSION

This study provides controlled empirical evidence for a decision that many frontend teams face: when to adopt a dedicated server-state management library. Query libraries provide measurable advantages in request deduplication, mutation lifecycle safety, and paginated data access, while offering no meaningful benefit for simple read scenarios. TanStack Query provides the most systematic support across complex scenarios; SWR is more compact for read-heavy use cases. Library adoption should be driven by application complexity and mutation requirements—not by default assumptions about

library superiority or inferiority. Ultimately, our results suggest that server-state management decisions are not merely implementation choices, but architectural decisions that directly impact system correctness and long-term maintainability.

IX. ACKNOWLEDGMENTS

The authors used AI-assisted writing tools (Claude by Anthropic and ChatGPT by OpenAI) for language refinement, wording suggestions, and editorial support. All technical content, experimental design, analysis, and final manuscript decisions were developed and verified by the authors.

REFERENCES

- [1] TanStack, “TanStack Query: Powerful asynchronous state management for TypeScript and JavaScript,” 2024. [Online]. Available: <https://tanstack.com/query/latest>
- [2] Meta Platforms, Inc., “useEffect – React,” 2024. [Online]. Available: <https://react.dev/reference/react/useEffect>
- [3] Vercel, Inc., “SWR: React Hooks for data fetching,” 2024. [Online]. Available: <https://swr.vercel.app/>
- [4] ISO/IEC, “ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models,” International Organization for Standardization, 2023.
- [5] M. Nottingham, “HTTP cache-control extensions for stale content,” RFC 5861, 2010. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc5861>
- [6] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *J. Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [8] R. N. Diniz-Junior, F. S. Lopes, and D. B. Claro, “Evaluating the performance of web rendering technologies based on JavaScript: Angular, React, and Vue,” in *Proc. XLVIII Latin American Computer Conf. (CLEI)*, 2022, pp. 1–9.